Санкт-Петербургский государственный университет

Системное программирование

Группа 24.М71-мм

Реализация многопоточной версии алгоритма FastADC в платформе Desbordante

МОРОЗКО Иван Дмитриевич

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
ассистент кафедры информационно-аналитических систем Г. А. Чернышев

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **5**

2. Обзор **6**

2.1. Базовые определения 6

2.2. Обзор существующих алгоритмов 7

2.3. Платформа Desbordante 9

3. Алгоритм FastADC **10**

3.1. Пространство предикатов 10

3.2. Набор подсказок и свидетельств 11

3.3. Параллельное построение набора подсказок 13

3.4. Приближённые покрытия набора свидетельств 14

3.5. Алгоритм AEI 15

4. Реализация **18**

4.1. Пространство предикатов 18

4.2. Индекс списка позиций (PLI) 18

4.3. Сегментирование индекса списка позиций 19

4.4. Набор подсказок и свидетельств 20

4.5. Параллельное построение набора подсказок 20

4.6. Оптимизация представления подсказок 21

4.7. AEI 22

4.8. Диаграмма классов 23

5. Результаты **25**

Заключение **31**

Список литературы **32**

Введение

В последние годы все большее значение приобретают методы обработки и анализа данных. К сожалению, данные, основанные на экспериментах из реального мира, зачастую оказываются неидеальными. В них могут содержаться ошибки, такие как пропущенные значения, неправильные значения и т. п. Недостоверная информация может привести к принятию неверного решения, поэтому очистке данных в современном мире уделяется особое значение [18].

В этом вопросе может помочь профилирование данных — процесс извлечения метаданных из набора данных. Методы профилирования, представляющие собой поиск некоторой формально описанной закономерности или правила будем называть примитивами. Например, одними из самых известных примитивов являются функциональные зависимости (ФЗ) [12]. Благодаря профилированию данных можно узнать внутренние закономерности в данных, и, благодаря ним, сделать выводы о возможных ошибках [8].

Одним из таких примитивов являются запрещающие ограничения (ЗО). Данный примитив обобщает и расширяет существующий класс ограничений для очистки данных [6]. Каждое ЗО представляет собой отношение между предикатами, указывающее, какие комбинации значений атрибутов являются несогласованными. Автоматических поиск таких закономерностей является трудоёмкой задачей [1], однако, ЗО обладают рядом преимуществ, по сравнению с другими примитивам. Во-первых, ЗО позволяют выражать ограничения, не способные быть выраженными другими примитивами. Во-вторых, в недавних исследованиях, ЗО использовались в качестве стандартного языка для описания ограничений данных [7, 10, 13]. Например, HOLOCLEAN [13] — это инструмент для очистки данных, входными данными которого являются наборы приближённых ЗО.

В рамках предыдущей работы [23] мною был реализован и интегрирован в открытую платформу профилирования данных Desbordante [9] алгоритм поиска приближённых запрещающих ограничений — FastADC.

Данная реализация представляла собой однопоточный вариант алгоритма и была высокопроизводительной альтернативой оригинальной реализации на языке Java [11]. Было получено ускорение по времени в среднем в 1.81 раз и экономия по памяти в среднем в 4.05 раза.

Однако в ходе экспериментов было выявлено, что построение набора подсказок занимает значительную часть общего времени работы алгоритма, в среднем 88.9%. Это наблюдение согласуется с результатами оригинальной работы, в которой авторы отмечают: “Building evidence set is expensive, with a cost quadratic in the number of tuples of r. Hence, parallel computation techniques are developed to further improve the efficiency” [11]. Действительно, сложность построения набора подсказок квадратична числу кортежей, что делает данный этап узким местом при обработке больших объёмов данных.

Параллельные вычисления являются эффективным способом ускорения ресурсоёмких алгоритмов [2]. Распределение вычислительной нагрузки между несколькими потоками позволяет существенно сократить время выполнения задач, особенно на современных многоядерных процессорах. Стоит отметить, что в оригинальной реализации авторами уже была реализована многопоточная версия алгоритма.

В связи с этим было принято решение о реализации многопоточной версии алгоритма FastADC в рамках платформы Desbordante. Это позволит более эффективно использовать вычислительные ресурсы и ускорить процесс поиска приближённых запрещающих ограничений, тем самым повысить удобство использования инструмента для конечных пользователей.

1. Постановка задачи

Целью данной учебной практики является реализация многопоточной версии алгоритма FastADC поиска приближенных запрещающих ограничений в платформе Desbordante. Для её достижения в прошлом семестре были выполнены следующие задачи:

1. Провести обзор предметной области
2. Провести сравнение существующих алгоритмов поиска ЗО
3. Провести обзор алгоритма FastADC
4. Реализовать однопоточную версию алгоритма в платформе Desbordante

Поскольку текущая учебная практика является продолжением предыдущей, в этом семестре были поставлены следующие задачи:

1. Расширить обзор алгоритма FastADC, добавив описание многопоточной версии
2. Реализовать многопоточную версию алгоритма в платформе Desbordante
3. Сравнить производительность реализованного алгоритма с оригинальной реализацией [19]
4. Провести сравнение производительности однопоточной и многопоточной реализаций алгоритма

2. Обзор

2.1. Базовые определения

Опишем основные понятия, использующиеся для описания запрещающих ограничений и алгоритма их поиска.

Используются стандартные определения из реляционной модели данных: R — отношение; $r \in R$ — атрибут; $s,\ t \in r$ — кортежи.

Определение 1 (Предикат)**.** Запрещающие ограничения определяются на основе предикатов. Каждый предикат p имеет форму $t.A_i\, op\, s.A_j$, где $t, s \in r$, $t \neq s$ (различные кортежи), $A_i, A_j \in R$, и $op \in \{<, \leq, >, \geq, =, \neq\}$. Сравнение проводится по одному и тому же атрибуту, если $i = j$. В противном случае, оно проводится между двумя атрибутами.

Определение 2 (Запрещающие ограничения)**.** ЗО φ на атрибуте r определяется следующим образом: $\forall t, s \in r$, $\neg(p_1 \wedge \ldots \wedge p_m)$, где $p_1, \ldots, p_m$ являются предикатами для кортежей t, s. Мы говорим, что φ удерживается на r, если для каждой пары кортежей $(t, s) \in r$ по крайней мере один из $p_1, \ldots, p_m$ не удовлетворяется. Пара кортежей (t, s) нарушает φ, если пара удовлетворяет каждому $p_i\ : i \in [1, m]$.

Определение 3 (Мера ошибки)**.** Значение меры ошибки g_1 для данного ЗО φ на r, обозначаемое как $g_1(\varphi, r)$, определяется как отношение числа пар кортежей, нарушающих φ, к общему числу пар кортежей в r. То есть, $g_1(\varphi, r) = \frac{|\{(t,s) | (t,s) \in r^2 \wedge (t,s) \text{ нарушает } \varphi\}|}{|r|^2 - |r|}$.

Определение 4 (Приближённое запрещающее ограничение)**.** При заданном пороге ошибки ϵ, φ является приближённым ЗО на r, если и только если $g_1(\varphi, r) \leq \epsilon$.

Определение 5 (Минимальное запрещающее ограничение)**.** ЗО φ является минимальным, если не существует отличного от φ ограничения φ', такого что (a) множество предикатов φ' является строгим подмножеством множества предикатов φ, и (b) φ' является приближённым ЗО на r.

Таблица 1: Пример таблицы.

A	B	C
1	2	фрукт
2	1	овощ
2	4	овощ

Приведём несколько примеров ЗО, удерживающихся на таблице 1.

Примитив (a) функциональная зависимость $A \to C$ может показать, что любые два кортежа, совпадающие на атрибуте A, совпадают на атрибуте C. Примитив (b) уникальная комбинация колонок (Unique Column Combination, UCC) может показать, что проекция по атрибутам A и B не имеет не имеет одинаковых кортежей. Оба этих примитива можно выразить в терминах ЗО:

(a) $\forall t, s \in r,\ \neg(t.A = s.A \wedge t.C \neq s.C)$

(b) $\forall t, s \in r,\ \neg(t.A = s.A \wedge t.B = s.B)$

(c) $\forall t, s \in r,\ \neg(t.A > s.B \wedge t.B \leq s.B)$

Так же, ЗО (c) показывает, что возможность использования не только операторов равенства/неравенства, но и числового сравнения, позволяет выражать и более сложные зависимости между данными.

2.2. Обзор существующих алгоритмов

Разработка алгоритмов поиска обычных и приближённых запрещающих ограничений ведётся с 2013 года с представления алгоритма FastDC [5], и постоянно открываются новые методы. Для оценки существующих алгоритмов поиска ЗО и выбора подходящего для реализации алгоритма в платформе Desbordante, было выделено несколько критериев отбора.

Критерии отбора

- Возможность поиска приближённых ЗО

  Несмотря на то, что нахождение приближённых ЗО является значительно более трудоёмкой задачей, именно они используются при очистке данных, в отличие от точных ЗО [6, 20].

- Возможность поиска гетерогенных ЗО

  Возможность использовать предикаты между различными атрибутами позволит увеличить объём информации, которую мы способны получить из данных.

- Нахождение минимальных ЗО

  Нахождение минимальных запрещающих ограничений предпочтительно, поскольку оно позволяет избежать избыточности. Алгоритмы, находящие все ЗО, во-первых, используют больше памяти, а во вторых, затрудняют очистку данных. Если ЗО φ удерживается на r, и существует ЗО ξ, следующее из φ, то в проверке ξ при очистке данных нет смысла.

- Наличие open-source реализации

  Наличие реализации с открытым кодом упрощает процесс реализации алгоритма на языке C++, и позволяет сравнить полученное решение с оригинальным.

- Эффективность по сравнению с предыдущими

  Самый эффективный алгоритм является и самым предпочтительным для реализации.

Оценка рассмотренных алгоритмов представлена на таблице 2. Алгоритмы представлены в порядке их выхода, от самого старого на первой строке, до state-of-the-art на последних.

Таблица 2: Оценка рассмотренных алгоритмов.

Алгоритм	Год	Приближённые ЗО	Гетерогенные ЗО	Минимальные ЗО	Открытый код	Эффективнее предыдущих	Константные ЗО
FastDC [5]	2013	−	+	+	+	+	+
A-FastDC [5]	2013	+	+	+	+	−	+
Hydra [4]	2017	−	+	+	+	+	−
BFastDC [16]	2018	+	+	+	−	+	+
DCFinder [17]	2019	+	+	+	+	+	−
ADCMiner [3]	2020	+	+	+	+	−	−
DCFinder+ [15]	2022	+	+	+	+*	+	−
FastADC [11]	2022	+	+	+	+	+	−

Возможность поиска ЗО, использующих в качестве одного из аргументов некое константное значение, была исследована, однако она не представляет должного интереса чтобы отметить её как необходимую. Данная колонка была помечена тёмно–зелёным цветом как не влияющая на выбор алгоритма.

Хотя авторами работы DCFinder+ была предоставлена открытая реализация[1], она, к сожалению, не поддерживает возможность поиска приближённых ЗО, описанную в статье. Таким образом, алгоритмами, удовлетворяющим всем критериям, стали DCFinder и FastADC. Однако последний является state-of-the-art алгоритмом. Показано, что FastADC быстрее DCFinder на порядок [11], вследствие чего и был выбран этот алгоритм.

2.3. Платформа Desbordante

Desbordante — это высокопроизводительный инструмент для профилирования данных с открытым исходным кодом, разработанный на C++ [9]. За счет использования низкоуровневых оптимизаций, таких как векторизация вычислений и специализированные аллокаторы, Desbordante обладает высокой производительностью.

Этот инструмент обладает удобным пользовательским интерфейсом, включая Python-привязки и frontend-клиент.

В настоящее время Desbordante активно развивается, постоянно пополняясь реализациями новых примитивов.

[1] https://github.com/eduardopena/fdcd

3. Алгоритм FastADC

На текущий момент все алгоритмы поиска приближённых ЗО следуют схеме, состоящей из двух этапов [5, 17, 15, 11]. На первом этапе по данному отношению R строится структура, называемая *набором свидетельств (evidence set, HC)*, а на втором этапе происходит поиск приближённых покрытий HC (подробности в разделе 3.4).

FastADC следует такой же схеме, но предлагает новое решение для построения HC, сначала создавая так называемый *набор подсказок* (clue set), а затем преобразуя его в HC, и предоставляя новый метод для нахождения приближённых ЗО из набора свидетельств — *приближённая инверсия свидетельства (Approximate Evidence Inversion, AEI)*.

Принцип работы алгоритма FastADC можно рассмотреть на Рис. 1.

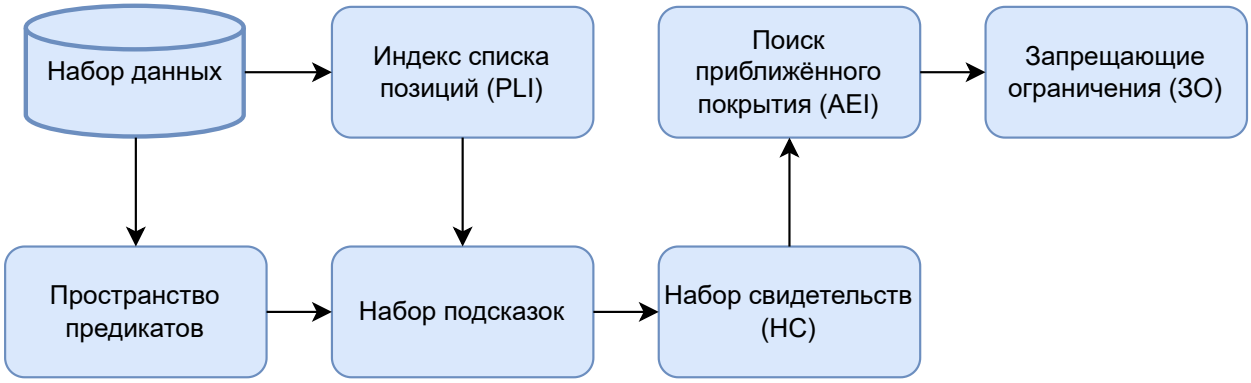

Рис. 1: Схема работы алгоритма FastADC.

3.1. Пространство предикатов

Первым этапом алгоритма FastADC служит построение так называемого пространства предикатов P — множества всех допустимых предикатов на R. Пространство предикатов строится, используя все операторы сравнения для числовых атрибутов, и операторы равенства/неравенства для категориальных. При этом используются только *сравнимые* атрибуты, т.е либо одинаковые, либо атрибуты одинакового типа, между которыми есть не меньше 30% общих значений, что позволяет избегать неинтересных предикатов.

Пример пространства предикатов можно увидеть на таблице 3. Данное пространство предикатов построено для таблицы 1, обсуждённой в разделе 2.1.

Таблица 3: Пространство предикатов для таблицы 1.

$p_1 : t.A = s.A$	$p_2 : t.A \neq s.A$
$p_3 : t.A > s.A$	$p_4 : t.A < s.A$
$p_5 : t.A \geq s.A$	$p_6 : t.A \leq s.A$
$p_7 : t.A = s.B$	$p_8 : t.A \neq s.B$
$p_9 : t.A > s.B$	$p_{10} : t.A < s.B$
$p_{11} : t.A \geq s.B$	$p_{12} : t.A \leq s.B$
$p_{13} : t.B = s.B$	$p_{14} : t.B \neq s.B$
$p_{15} : t.B > s.B$	$p_{16} : t.B < s.B$
$p_{17} : t.B \geq s.B$	$p_{18} : t.B \leq s.B$
$p_{19} : t.C = s.C$	$p_{20} : t.C \neq s.C$

3.2. Набор подсказок и свидетельств

Дадим определение набора свидетельств.

Определение 6 (Свидетельство)**.** Свидетельство $evi(t, s)$ для пары кортежей $(t, s) \in r^2$ представляет собой множество предикатов $p \subset P$, удовлетворяющихся на паре кортежей (t, s).

Определение 7 (Набор свидетельств)**.** НС определяется как множество свидетельств по всем парам кортежей: $\left\{evi(t, s) : (t, s) \in r^2\right\}$.

Обычно НС реализуется с помощью битовых шкал (bit set), что позволяет эффективно выполнять алгоритм поиска приближённых покрытий, описанный в разделе 3.4. Поскольку $evi(t, s) = \bigcup\limits_{A,B \in R} \{$предикаты $p : p = (t.A\ op\ s.B)\}$, где $op \in \{=, \neq\}$, либо $\in \{=, \neq, <, >, \geq, \leq\}$, то для кодирования предиката для каждой пары категориальных атрибутов используется два бита, а для пары числовых атрибутов — шесть [16, 17].

Однако построение НС из пространства предикатов с данным подходом может быть затруднительно. Как будет показано далее, FastADC строит НС гораздо быстрее существующих алгоритмов, ввиду введения

новой структуры — набора подсказок. Подсказка $clue(t, s)$ кодирует отношения между кортежами сравнимых пар атрибутов более сжато, чем набор свидетельств, требует меньше памяти для хранения, и в то же время биективна $evi(t, s)$.

$clue(t, s)$, как и $evi(t, s)$, тоже описывает все предикаты, удовлетворяющиеся на паре кортежей (t, s), однако кодирует **одним** битом предикаты для категориальной пары атрибутов: 0, если $t.A \neq s.B$ (или $t.A \neq s.A$), или 1, если $t.A = s.B$ (или $t.A = s.A$); и **двумя** битами для численных атрибутов: 00, если $t.A < s.B$ (или $t.A < s.A$), 01, если $t.A = s.B$ (или $t.A = s.A$), или 10, если $t.A > s.B$ (или $t.A > s.A$).

Набор подсказок строится следующим образом. Изначально каждой паре кортежей присваивается начальная подсказка c_0, после чего подсказки корректируются для пар кортежей, чьи подсказки отличаются от начальной. Эффективность этого процесса достигается за счёт использования индекса списка позиций (Position List Index, PLI)[2] отношения r. Такой подход значительно эффективнее стандартного метода, сравнивающего все пары кортежей.

Выбор c_0 влияет на эффективность: меньшее количество бит требует коррекции, если c_0 имеет много общих бит с фактическими подсказками. Исходя из предположения что большинство кортежей имеют различные значения атрибутов [17], c_0 выбирается как битовая шкала со всеми значениями равными нулю, т.е., для пары кортежей (t, s) и пары атрибутов (A, B) предполагаем, что $t.A \neq s.B$ для категориальных пар атрибутов и $t.A < s.B$ для числовых пар.

Например, в таблице 1 сравнимыми парами атрибутов являются пары $(A, A), (C, C), (A, B)$ и (B, B). Тогда подсказка $clue(2, 3)$ будет выглядеть следующим образом:

$$clue(2,3) = \underbrace{01}_{\text{AA}}\ \underbrace{1}_{\text{CC}}\ \underbrace{00}_{\text{AB}}\ \underbrace{00}_{\text{BB}}$$

что соответствует предикатам $\{p_1, p_{19}, p_{10}, p_{16}\}$, соответственно.

В худшем случае сложность алгоритма квадратична по отношению

[2]Подробнее про PLI написано в разделе 4.2.

к $|r|$, но на практике многие пары кортежей могут быть пропущены благодаря PLI. Также скорость данного алгоритма, в частности, достигается за счёт попадания подсказок в кеш процессора. Каждая подсказка может быть скорректирована несколько раз, при этом при коррекции модифицируется максимум один бит для каждой пары атрибутов: бит 0 может быть изменён только на 1 для категориальных пар атрибутов, а 00 может быть изменён только на 01 или 10 для числовых пар.

Генерация доказательств из подсказок происходит через простое преобразование битов [11]. Сложность этого линейна по отношению к размеру набора подсказок, а мощность этого множества $\ll |r|^2$, поскольку многие пары кортежей могут производить одинаковые подсказки, как было показано авторами работы [11]. Тем самым, время преобразования незначительно по сравнению с временем создания набора подсказок.

3.3. Параллельное построение набора подсказок

На практике именно этот этап занимает значительную часть общего времени работы алгоритма. Авторы оригинальной работы [11] отмечают, что параллельные вычисления позволяют существенно ускорить данный процесс.

Наблюдение, делающее параллелизацию возможной, заключается в том, что вычисление подсказки для конкретной пары кортежей зависит только от значений этих кортежей и не требует информации о других парах. Это позволяет разделить отношение r на несколько частей, вычислить подсказки на этих частях и затем объединить частичные результаты.

Для организации параллельных вычислений отношение r разбивается на части, называемые сегментами. Размер сегмента является настраиваемым параметром алгоритма, по умолчанию составляющим 350 строк. Подробнее механизм сегментирования описан в разделе 4.3.

Для каждой пары сегментов r_i и r_j можно независимо построить

частичный набор подсказок:

$$clue_{ij} = \{clue(t, s) \mid t \in r_i, s \in r_j\}$$

Для построения всех частичных наборов подсказок необходимо рассмотреть все возможные пары сегментов. При n сегментах это пары (r_i, r_j) для всех $0 \leq i \leq j < n$. Условие $i \leq j$ исключает повторную обработку одних и тех же пар кортежей: пара кортежей (t, s), где $t \in r_i$ и $s \in r_j$, обрабатывается только при построении $clue_{ij}$, но не $clue_{ji}$. Всего таких пар сегментов $\frac{n(n+1)}{2}$, что и определяет количество независимых задач для параллельного выполнения. Каждая пара кортежей обрабатывается ровно одним потоком, что исключает дублирование вычислений. Итоговый набор подсказок получается объединением частичных наборов.

3.4. Приближённые покрытия набора свидетельств

Для начала определим понятие обычного и приближенного покрытия НС.

Определение 8 (Покрытие НС)**.** Покрытие НС — это подмножество X множества P, такое что $\forall e \in$ НС $X \cap e \neq \emptyset$.

Авторы статьи [5], положившей начало алгоритмам поиска приближённых запрещающих ограничений, ввели и доказали следующую теорему:

Теорема 1 (Допустимость точного ЗО)**.** ЗО $\varphi = \neg(\overline{p_1} \wedge \cdots \wedge \overline{p_m})$ допустимо на r, тогда и только тогда, когда множество $\{p_1, \ldots, p_m\}$ является покрытием НС.

С помощью данной теоремы находятся точные запрещающие ограничения. Авторы той же работы [5] также расширяют это определение для приближённых ЗО.

Определение 9 (Функция cnt)**.** Разные пары кортежей могут производить одинаковые свидетельства. Функция $cnt(e)$ для свидетельства e обозначает число пар кортежей (t, s), таких что $evi(t, s) = e$.

Определение 10 (Приближённое покрытие НС)**.** Приближенное покрытие НС это такое подмножество $X \subset P$, что для всех $e \in$ НС, таких что $X \cap e \neq \emptyset$, $\sum cnt(e) \geq (1-\epsilon) \times (|r|^2 - |r|)$.

Теорема 2 (Валидность приближённого ЗО)**.** При данном пороге ошибки ϵ, приближённое ЗО $\varphi = \neg(\overline{p_1} \wedge \cdots \wedge \overline{p_m})$ допустимо на r, тогда и только тогда, когда $\{p_1, \ldots, p_m\}$ является приближённым покрытием НС.

Иными словами, множество инвертированных предикатов ЗО должно пересекаться с «достаточным» количеством свидетельств (не обязательно со всеми).

Прежде чем перейти к методу приближенной инверсии доказательств (AEI), введем некоторые обозначения.

- Для ЗО $\phi = \neg(p_1 \wedge \ldots \wedge p_m)$ и свидетельства e, $\phi \subseteq e$, означает что набор предикатов ϕ, т.е., $\{p_1, \ldots, p_m\}$, является подмножеством e. Очевидное следствие: $\phi \subseteq e(t,s) \Rightarrow \phi$ нарушается парой кортежей (t,s) (но не обязательно наоборот).

- Говорим, что ЗО ϕ *покрывает* $e(t,s)$, если $\phi \nsubseteq e$, и ϕ не нарушается парой кортежей (t,s).

- Для двух ЗО ϕ и ϕ' мы пишем $\phi \subseteq \phi'$, если набор предикатов ϕ является подмножеством набора предикатов ϕ'. Очевидное следствие: ϕ' не является минимальным, если $\exists \phi : \phi \subsetneq \phi'$ и ϕ действителен.

- $\phi \cup \{p\}$ обозначает ЗО, получаемый добавлением нового предиката p к ϕ. Данный процесс называется *уточнением* ϕ.

3.5. Алгоритм AEI

Основная идея алгоритма AEI выглядит следующим образом. Определятся начальный набор ЗО Σ, который постепенно модифицируется, и в итоге становится ответом. Этот набор инициализируется запрещающими ограничениями, состоящими из одного предиката:

$\Sigma \overset{init}{=} \{\varphi : \varphi = \neg(p)\ \forall p \in P\}$. На каждой итерации алгоритма обрабатывается одно свидетельство e из НС следующим образом: для каждого $\varphi \in \Sigma$ проверяется, *покрывает* ли оно e. Если φ *не покрывает* свидетельство, алгоритм *пытается покрыть* φ e: к φ добавляются новые предикаты из P, которые не входят в e, создавая новые кандидатные ЗО. После каждого добавления нового предиката к φ проверяется, остается ли полученное ЗО минимальным. Если нет, ЗО исключается дальнейшего рассмотрения. Итерации повторяются пока каждое свидетельство не будет рассмотрено.

Однако данное описание сможет найти только точные ЗО, поскольку при генерации кандидатов приближенных ЗО необязательно разрешать все нарушения ЗО относительно каждого свидетельства e. Вместо этого должно учитываться количество ($cnt(e)$) для каждого e при генерации или отсечении кандидатов.

Модификации для нахождения приближённых ЗО

Каждое кандидатное ЗО φ хранится в паре с множеством предикатов $cand$, которые могут быть использованы для *уточнения* φ. Множество таких пар называется Ψ.

Как и раньше, на каждой итерации рассматривается свидетельство $e \in$ НС. Все кандидатные ЗО из Ψ делятся на две части в зависимости от того, покрывают ли они e. ЗО которые не покрывают e, помещаются в Ψ^-, в то время как остальные остаются в Ψ.

Затем AEI пытается покрыть ЗО $\varphi \in \Psi^-$, как и ранее описанный алгоритм, но в этот раз сначала с использованием cnt проверяется, может ли этот путь привести к корректному ЗО. Вычисляется количество оставшихся свидетельств, которые могут быть покрыты оставшимися предикатами, т.е. верхняя граница накопленного количества, и данное число сравнивается с N — суммарным числом свидетельств, которые обязаны быть покрытыми (инициализируется выражением $(1 - \epsilon) \times (|r|^2 - |r|)$ из определения 10). Если полученное число меньше N, то из $cand$ удаляется e. Если после этого $cand$ представляет собой пустое множество, то φ больше невозможно *уточнять*. Происходит проверка

того что φ является минимальным и приближённым ЗО, и φ добавляется в Σ. Происходит рекурсивный вызов, которому передаётся Ψ^- и следующее свидетельство.

В ветви, где e покрыто, собирается отдельное множество предикатов $\{\bigcup_{\{_,cand\}\in\Psi^-} cand \setminus e\}$. Для каждого предиката p из этого множества формируется новое кандидатное ЗО путём *уточнения* $\varphi \in \Psi$: $\varphi' = \varphi \cup \{p\}$, и определяется набор $cand'$ с предикатами из $cand$, определённых на <u>тех же</u> парах атрибутов что и p. Если такие предикаты существуют, то Ψ обновляется — в него добавляется новая пара $\{\varphi', cand'\}$, если, конечно, φ' минимальна в Ψ, т.е $\nexists \varphi \in \Psi : \varphi \subseteq \varphi'$.

Если же $cand'$ представляет собой пустое множество, происходит то же самое что и в первой ветви: проверка того, что φ' является минимальным и приближённым ЗО; добавление φ' в Σ.

Обе ветви пройдены. N уменьшается на $cnt(e)$ и метод рекурсивно вызывается, в этот раз для Ψ.

Более подробно алгоритм AEI можно рассмотреть в работе [11].

4. Реализация

Были реализованы необходимые классы для построения пространства предикатов, индекса списка позиций, набора подсказок, набора свидетельств и приближенных инверсий свидетельств.

Рассмотрим ближе отдельные блоки алгоритма.

4.1. Пространство предикатов

Пространство предикатов представляет собой список классов *Predicate*. Для построения класса *Predicate*, описывающего предикат $p = t.A\, op\, s.B$, были построены классы *ColumnOperand* и *Operator*.

Алгоритм построения пространства предикатов проходится по каждой паре атрибутов и вычисляет, являются ли атрибуты сравнимыми. Напомним, что *сравнимые* атрибуты, это либо одинаковые атрибуты, либо атрибуты одинакового типа, между которыми есть не меньше 30% общих значений. Были реализованы необходимые функции, возвращающие метрику сравнимости для атрибутов категориальных и числовых типов.

4.2. Индекс списка позиций (PLI)

Каждый PLI строится для атрибута $A \in R$. При этом предварительно значения атрибута хешируются, что позволяет говорить о значении кортежа на атрибуте A как об обычном числе — ключе, не заботясь о настоящем типе атрибута.

Определение 11 (Кластер)**.** Кластером в контексте PLI мы называем пару $\langle k, l \rangle$, где ключ k является значением в атрибуте A, а l — множество всех кортежей, имеющих то же значение k в атрибуте A.

Определение 12 (Индекс списка позиций (PLI))**.** Для атрибута A, класс PLI является тройкой:

- Множество ключей уникальных значений атрибута A
- Список кластеров
- Отображение ключ $\rightarrow$ кластер

Для числового атрибута A, кластеры в PLI дополнительно сортируются по ключу k в порядке убывания, что позволит оптимизировать построение набора подсказок путём использования двоичного поиска.

Данный класс был реализован, наряду с методом хеширования значений атрибутов таблицы.

4.3. Сегментирование индекса списка позиций

Для организации параллельного построения набора подсказок исходное отношение разбивается на части фиксированного размера, называемые сегментами. Каждый сегмент содержит кортежи с последовательными идентификаторами. Размер сегмента является конфигурируемым параметром алгоритма, по умолчанию составляющим 350 строк. Например, первый сегмент содержит строки с 0 по 349, второй — с 350 по 699 и т.д.

Определение 13 (PliShard)**.** PliShard представляет собой набор PLI, построенных на кортежах одного сегмента. Для каждого атрибута отношения PliShard содержит отдельный PLI, учитывающий только кортежи данного сегмента.

Класс *PliShardBuilder* выполняет разбиение отношения на сегменты и построение соответствующих PLI. Суммарная временная и пространственная сложность построения всех PliShard совпадает со сложностью построения PLI для всего отношения, поскольку каждый кортеж обрабатывается ровно один раз.

4.4. Набор подсказок и свидетельств

Для построения набора подсказок были реализованы два вспомогательных класса *SingleClueSetBuilder* и *CrossClueSetBuilder*.

Класс *SingleClueSetBuilder* используется для построения частичного набора подсказок в случае, когда оба кортежа пары принадлежат одному сегменту. Он принимает на вход один PliShard и строит подсказки по алгоритму, описанному ранее в разделе 3.2.

Класс *CrossClueSetBuilder* используется для случая, когда кортежи принадлежат разным сегментам. Он принимает на вход два различных PliShard и модифицирует логику построения: при поиске совпадающих значений атрибутов кортеж из кластера первого сегмента сопоставляется с кортежами из соответствующего кластера второго сегмента. При этом заполняются два массива подсказок: для пар $(t \in r_i, s \in r_j)$ и для пар $(t \in r_j, s \in r_i)$.

4.5. Параллельное построение набора подсказок

Для параллельного построения набора подсказок была реализована функция *BuildClueSetParallel*, использующая набор исполнителей.

Задачи индексируются линейно от 0 до $\frac{n(n+1)}{2} - 1$, где n — число сегментов. Вспомогательная функция *TaskIdToIndices* преобразует линейный индекс задачи в пару индексов сегментов (i, j) с помощью двоичного поиска. Для минимизации накладных расходов на синхронизацию каждый рабочий поток владеет локальным массивом для набора подсказок.

Выполнение организовано по схеме map-reduce:

1. Каждая задача определяет пару сегментов (i, j) по своему индексу
2. В зависимости от того, совпадают ли индексы, создаётся *SingleClueSetBuilder* или *CrossClueSetBuilder*
3. Результат задачи добавляется в локальный массив с наборами подсказок

4. По завершении всех задач потока его локальный массив сливается в глобальный набор подсказок под защитой мьютекса

Набор свидетельств принимает в качестве входных данных набор подсказок и пространство предикатов, и строит свидетельства через простое преобразование битов [11].

4.6. Оптимизация представления подсказок

Как отмечалось в разделе 3.2, подсказка представляет собой битовую шкалу, где каждый бит соответствует определённой позиции в пространстве предикатов. Количество бит в подсказке определяется числом сравнимых пар атрибутов: для каждой категориальной пары требуется один бит, для каждой числовой — два бита.

В оригинальной реализации на Java для хранения подсказок используется тип `LongBitSet`, основанный на 64-битном целом числе[3]. Это накладывает ограничение: алгоритм не может обрабатывать наборы данных, для которых суммарное число бит в подсказке превышает 64.

В рамках данной работы это ограничение было ослаблено до 128 бит за счёт использования типа `std::bitset<128>`. Однако использование фиксированного 128-битного представления для всех случаев приводит к ряду проблем с производительностью. Во первых, при построении подсказок для пары сегментов алгоритм оперирует массивами размера, пропорционального квадрату размера сегмента. Элемент типа `std::bitset<128>` занимает 16 байт, тогда как `uint64_t` — 8 байт. Это увеличивает использование памяти и ухудшает эффективность использования кеша процессора. Во вторых, после построения набора подсказок алгоритм объединяет одинаковые подсказки в отображении "подсказка → количество". При использовании 128-битного ключа операции хеширования и сравнения затрагивают несколько машинных слов, что увеличивает время поиска и вставки.

[3] https://github.com/RangerShaw/FastADC/blob/master/src/main/java/FastADC/evidence/clue/ClueSetBuilder.java#L111

Для решения этих проблем была реализована оптимизация, основанная на следующем наблюдении: число бит в подсказке известно до начала построения набора подсказок, поскольку оно определяется пространством предикатов. Это позволяет выбрать минимально достаточный тип данных для хранения подсказки в зависимости от фактического числа используемых бит b:

- `uint8_t` при $b \leq 8$;
- `uint16_t` при $8 < b \leq 16$;
- `uint32_t` при $16 < b \leq 32$;
- `uint64_t` при $32 < b \leq 64$;
- `std::bitset<128>` при $64 < b \leq 128$.

Логика построения подсказок при этом остаётся неизменной: используются позиции бит, а операция установки бита реализуется единообразно для всех типов.

При числе бит больше 64 алгоритм вынужден использовать 128-битное представление. В этом случае выводится предупреждение о возможном снижении производительности. Если число бит превышает 128, алгоритм завершает работу с сообщением о том, что пространство предикатов слишком велико для текущей реализации.

4.7. AEI

Помимо классов *DenialConstaint* и *DenialConstaintSet*, нужных для представления итогового результата, для реализации алгоритма AEI, описанной в разделе 3.5, были реализованы вспомогательные классы *PredicateOrganizer*, *Closure* и *DCCandidateTrie*.

- *PredicateOrganizer*

  Данный класс отвечает за преобразование и упорядочивание предикатов для оптимизации процесса инверсии свидетельств.

Определение 14 (Покрытие предиката)**.** Покрытием предиката называется количество свидетельств, в которых присутствует данный предикат.

Для каждого предиката класс вычисляет его покрытие, и перестраивает порядок предикатов таким образом, чтобы предикаты с меньшим покрытием обрабатывались первыми. Это позволяет уменьшить размер пространства поиска и повысить эффективность AEI, так как потенциально проблемные предикаты будут рассматриваться в первую очередь.

- *Closure*

  Данный класс принимает множество предикатов в качестве входных данных, и добавляет к нему все предикаты которые можно получить из данного множества. Например, к предикатам $A < B$ и $B < C$, *Closure* добавит предикат $A < C$ как транзитивный, предикат $A \leq B$ как импликацию $A < B$, и все симметричные им. Это необходимо, поскольку все логически вытекающие предикаты должны быть учтены при проверке минимальности и валидности запрещающих ограничений.

- *DCCandidateTrie*

  Данный класс реализует префиксное дерево (trie) для эффективных операций над множеством кандидатов ЗО, таких как определение того, содержит ли дерево кандидат, являющийся подмножеством данного кандидата, или удаление из дерева всех кандидатов, являющихся обобщениями текущего свидетельства.

4.8. Диаграмма классов

На Рис. 2 представлена UML–диаграмма классов, описывающая структуру решения.

Синим цветом выделена основная функция построения набора запрещающих ограничений. Серым цветом выделена пояснительная за-

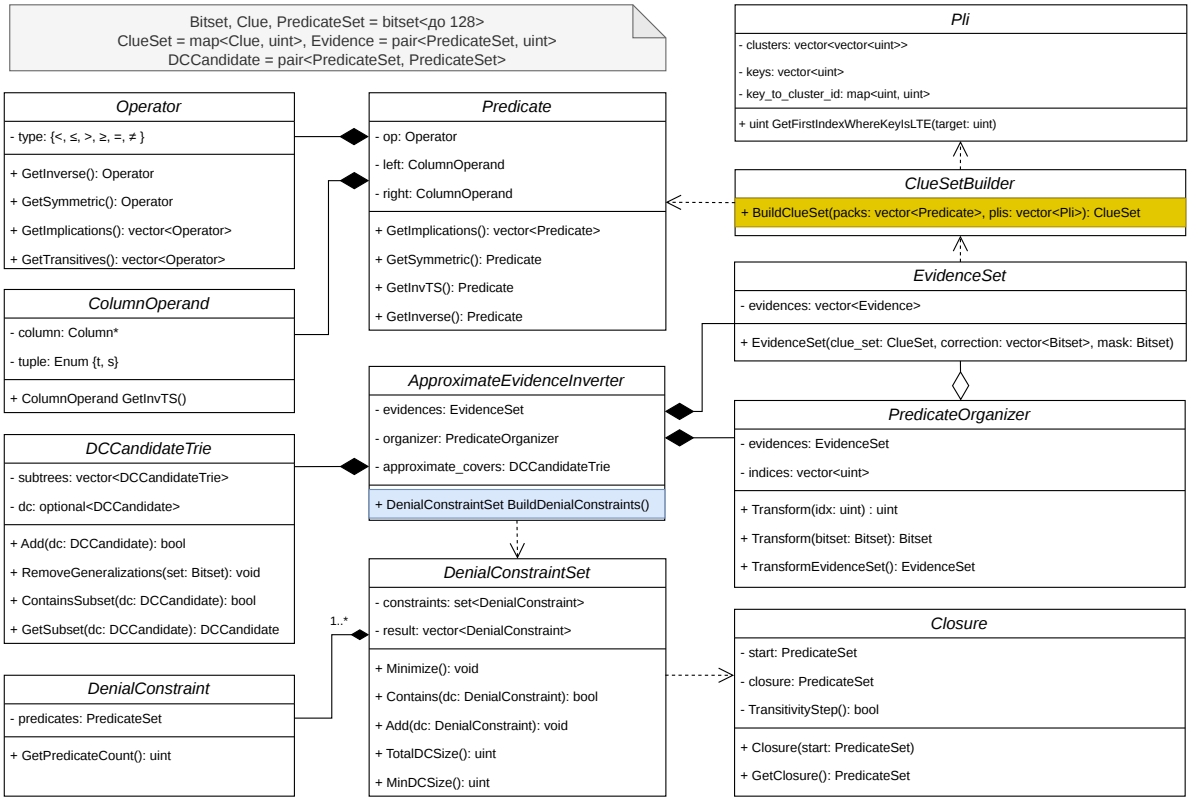

Рис. 2: UML–диаграмма классов алгоритма FastADC.

писка с указанием используемых псевдонимов. Жёлтым цветом выделена функция *BuildClueSet*, которая была модифицирована в рамках текущей работы: в зависимости от настраиваемого параметра числа потоков функция выбирает однопоточную или многопоточную реализацию построения набора подсказок.

5. Результаты

Для оценки производительности полученной реализации были проведены эксперименты. Измерялось время построения набора свидетельств, время работы алгоритма AEI, общее время выполнения и затраты памяти на наборах данных, представленных в таблице 4.

Обе реализации выводят время работы соответствующих этапов без учёта обработки входного файла, что использовалось в качестве показателя производительности по времени. Память же измерялась на отдельных запусках, где каждые 100 миллисекунд собиралась информация о затратах памяти командой `ps -o rss= -p $PID`. Для каждого набора данных проводилось десять запусков, из полученных данных вычислялось среднее значение с доверительным интервалом 95%.

Desbordante компилировался с флагом оптимизации `-O3`. Авторами оригинальной реализации не была предоставлена дополнительная информация о конфигурации виртуальной машины Java, поэтому была использована LTS версия JDK с настройками виртуальной машины по умолчанию.

При тестировании возник ряд особенностей, требующих дополнительного внимания. Современные процессоры Intel содержат два типа ядер: производительные ядра (P-cores) и энергоэффективные ядра (E-cores) [14]. P-cores обеспечивают максимальную производительность на поток, тогда как E-cores оптимизированы для фоновых задач с меньшим энергопотреблением. Смешение этих типов ядер при тестировании может исказить результаты измерений.

Кроме того, современные процессоры поддерживают технологию одновременной многопоточности (Simultaneous Multithreading, SMT), также известную как гиперпоточность, при которой физическое ядро определяется операционной системой как два отдельных логических ядра [22, 21]. Эта технология может повлиять на результаты производительности, поэтому эксперименты проводились в двух конфигурациях:

1. SMT отключён через интерфейс `/sys/devices/system/cpu/smt/control`. Использовалось по одно-

му потоку на каждое физическое P-core.

2. SMT включён. Использовались все логические ядра на P-cores и как следствие по два потока на каждое физическое ядро.

На тестовом стенде процессор содержит 6 производительных ядер. Таким образом, в конфигурации с выключенным SMT использовалось 6 потоков, а в конфигурации с включённым SMT — 12 потоков.

Тип ядра определялся через анализ файла `thread_siblings_list` в файловой системе `/sys`. Привязка процессов к выбранным ядрам осуществлялась командой `taskset`.

Для Java-реализации, использующей набор исполнителей ForkJoin для параллельного построения набора подсказок, явно задавался уровень параллелизма через параметры виртуальной машины:

- `-XX:ActiveProcessorCount` для указания числа доступных процессоров;
- `-Djava.util.concurrent.ForkJoinPool.common.parallelism` для задания размера общего набора исполнителей.

Эксперименты проводились на ненагруженном тестовом стенде, представленном в таблице 5, без других ресурсоёмких процессов. Для всех задействованных ядер устанавливался режим производительности через `cpupower -c $CORE frequency-set` с фиксацией максимальной частоты. Для минимизации влияния операционной системы отключался файл подкачки командой `swapoff -a`, а между запусками очищались кеши путём записи `3` в файл `/proc/sys/vm/drop_caches`.

Таблица 4: Характеристики наборов данных, использовавшихся для проведения экспериментов.

Название	**Атрибуты**	**Строки**	**Размер**	**# ЗО**
Airport	11	55113	5.74 Мб	122
Atom	13	147067	7.47 Мб	3835
Food	12	200000	71.80 Мб	65
SPStock	6	122496	4.02 Мб	315
Neighbors	7	10000	647.5 Кб	2236

Таблица 5: Характеристики тестового стенда.

Компонент	**Спецификация**
Процессор	12th Gen Intel Core i7-12800H, 4.7 GHz
Оперативная память	32GiB DDR5, 4800 MHz
Запоминающее устройство	NVMe SED Samsung 512GB
Операционная система	Ubuntu 24.04.3 LTS
Ядро Linux	6.8.0-94-generic
Версия OpenJDK	21.0.10 2026-01-20 LTS
Версия GCC	13.3.0

Результаты представлены на Рис. 3, Рис. 4, Рис. 5, Рис. 6 и Рис. 7.

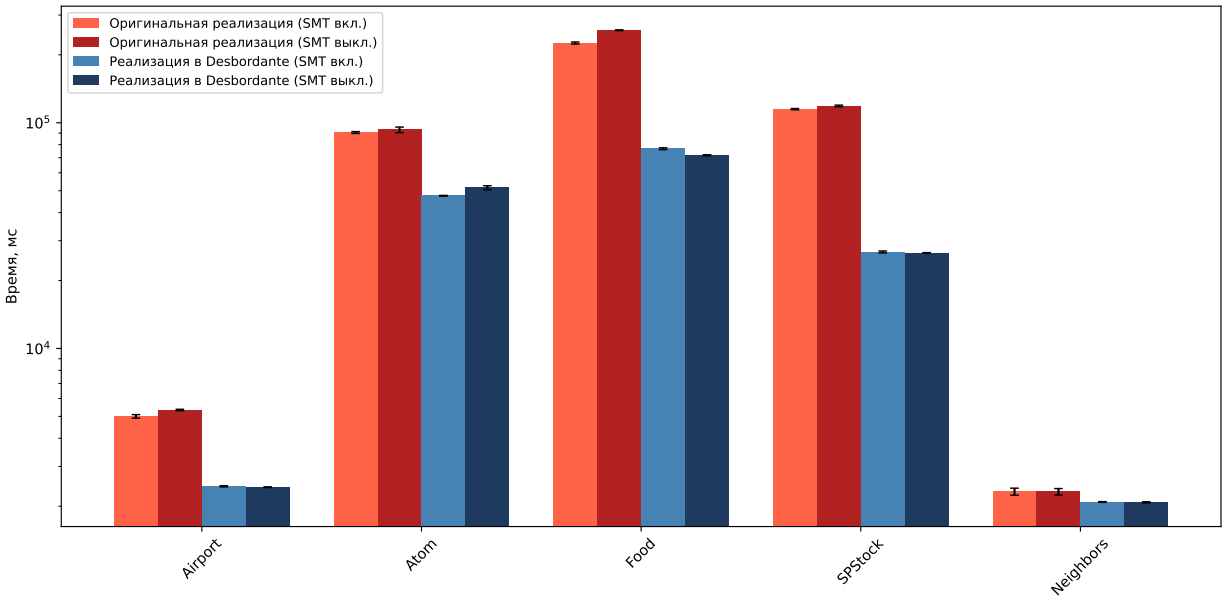

Рис. 3: Суммарное время выполнения FastADC.

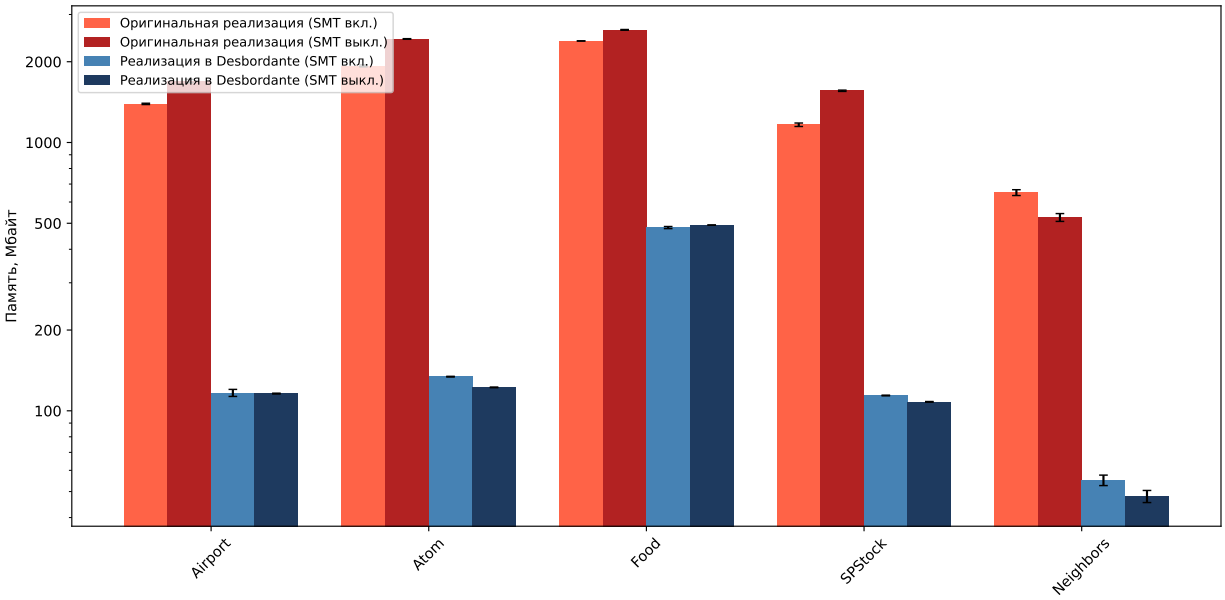

Рис. 4: Затраты памяти.

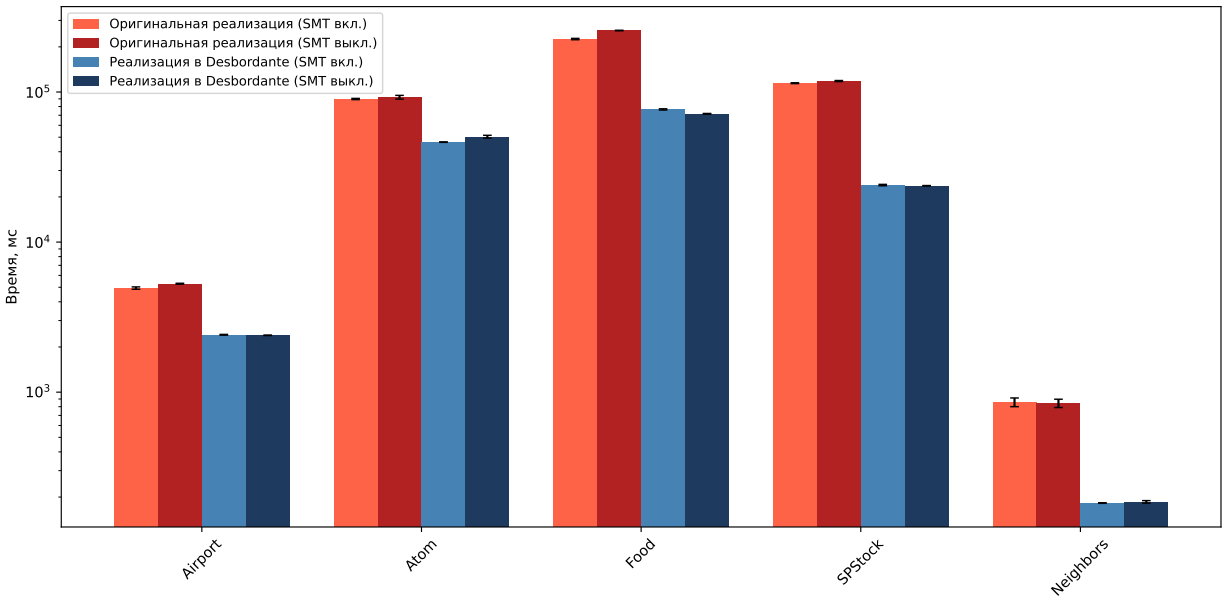

Рис. 5: Время построения набора свидетельств.

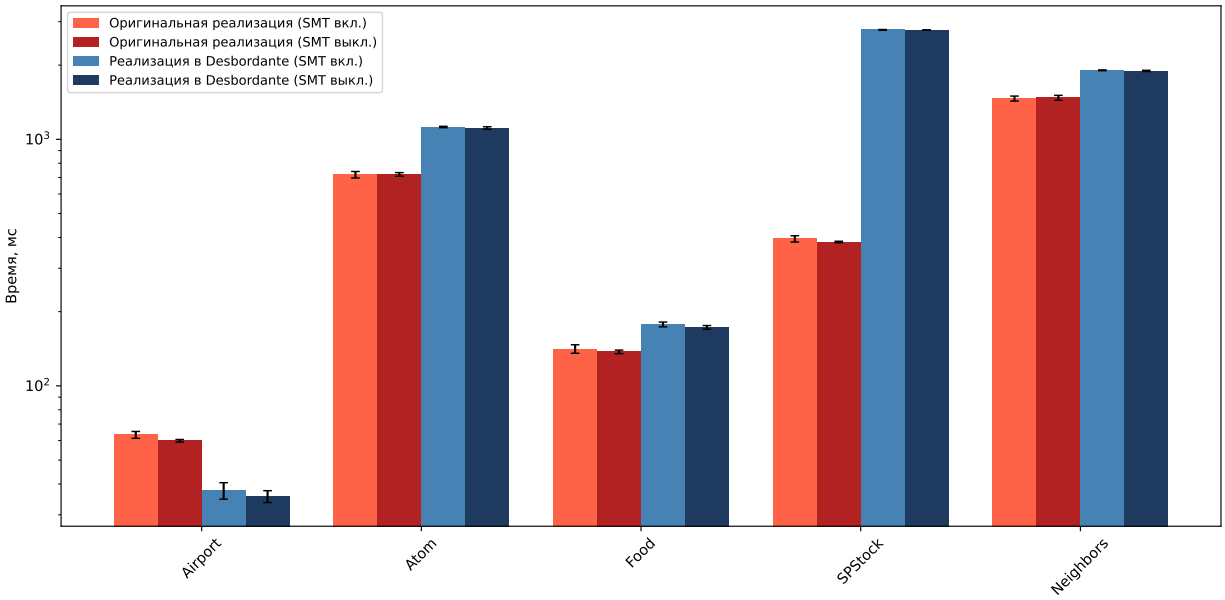

Рис. 6: Время выполнения алгоритма AEI.

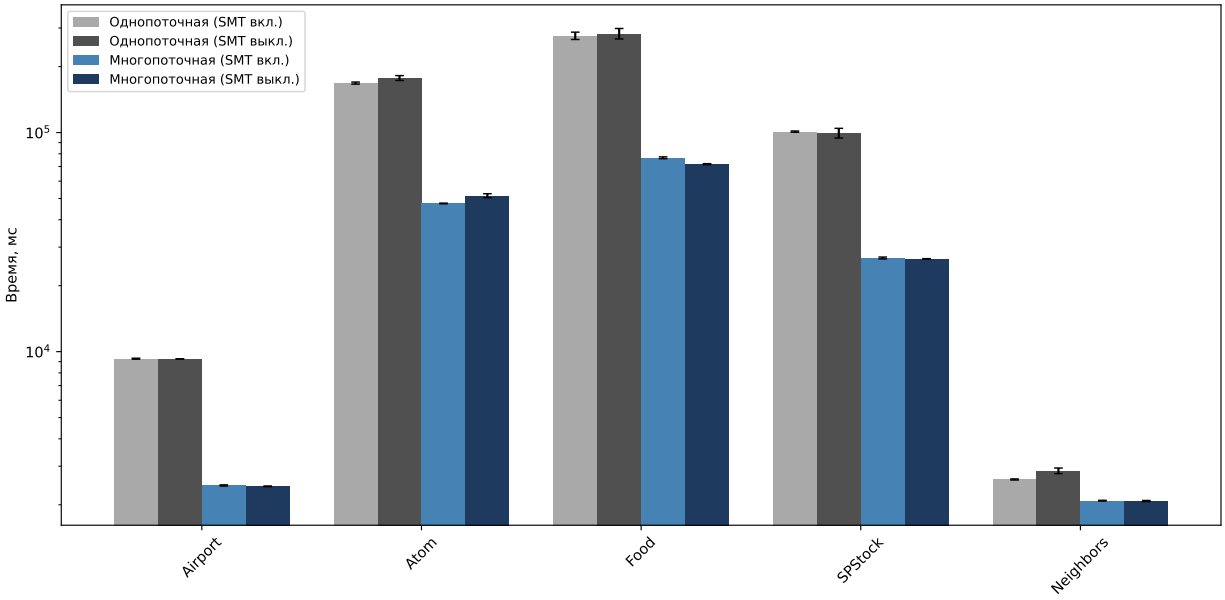

Рис. 7: Сравнение однопоточной и многопоточной версий Desbordante по суммарному времени выполнения.

Реализация в Desbordante показала ускорение по суммарному времени выполнения на всех пяти наборах данных. В конфигурации с выключенным SMT ускорение составило от 1.11 раза на наборе данных Neighbors до 4.48 раза на наборе данных SPStock. В среднем ускорение составило 2.64 раза. В конфигурации с включённым SMT результаты оказались схожими: среднее ускорение составило 2.46 раза.

При построении набора свидетельств ускорение составило от 1.83 раза на наборе данных Atom до 4.99 раза на наборе данных SPStock. В среднем ускорение составило 3.43 раза.

При выполнении алгоритма AEI реализация в Desbordante оказалась медленнее оригинальной на большинстве наборов данных. Однако, как отмечалось ранее, основную часть времени работы алгоритма занимает именно построение набора свидетельств, поэтому данное замедление не оказывает существенного влияния на общую производительность.

Экономия по памяти составила от 5.34 раза на наборе данных Food до 19.88 раза на наборе данных Atom. В среднем экономия составила 13.05 раза в конфигурации с выключенным SMT и 10.66 раза в конфигурации с включённым SMT.

Разница в производительности между конфигурациями с включённым и выключенным SMT оказалась незначительной для обеих реализаций.

Многопоточная реализация в Desbordante показала существенное ускорение по сравнению с однопоточной версией. По суммарному времени выполнения ускорение составило от 1.37 раза на наборе данных Neighbors до 3.94 раза на наборе данных Food. В среднем ускорение составило 3.27 раза.

Меньшее ускорение на наборе данных Neighbors объясняется тем, что значительную часть времени выполнения на этом наборе занимает алгоритм AEI, который в текущей реализации оказался медленнее оригинальной реализации на Java.

Заключение

В предыдущем семестре были выполнены следующие задачи:

- Проведён обзор предметной области
- Выбран набор критериев, по которым проведён обзор алгоритмов поиска ЗО
- Проведён обзор алгоритма FastADC
- Реализована однопоточная версия алгоритма FastADC в платформе Desbordante

В результате текущей проделанной работы были выполнены следующие задачи:

- Расширен обзор алгоритма FastADC, добавлено описание многопоточной версии
- Реализована многопоточная версия алгоритма FastADC в платформе Desbordante
- Проведено сравнение производительности с оригинальной реализацией на Java
  - Получено ускорение по времени в среднем в 2.64 раза
  - Получена экономия по памяти в среднем в 13.05 раза
- Проведено сравнение однопоточной и многопоточной версий
  - Получено ускорение в среднем в 3.27 раза

Код находится в репозитории[4].

[4] https://github.com/Desbordante/desbordante-core/pull/695

Список литературы

[1] Abedjan Ziawasch, Golab Lukasz, Naumann Felix. Profiling relational data: a survey // The VLDB Journal. — 2015. — Vol. 24, no. 4. — P. 557–581. — URL: https://doi.org/10.1007/s00778-015-0389-y.

[2] Amdahl Gene M. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities // Proceedings of the AFIPS Spring Joint Computer Conference. — 1967. — P. 483–485.

[3] Livshits Ester, Heidari Alireza, Ilyas Ihab F., Kimelfeld Benny. Approximate Denial Constraints. — 2020. — 2005.08540.

[4] Bleifuß Tobias, Kruse Sebastian, Naumann Felix. Efficient denial constraint discovery with hydra // Proc. VLDB Endow. — 2017. — Vol. 11, no. 3. — P. 311–323. — URL: https://doi.org/10.14778/3157794.3157800.

[5] Chu Xu, Ilyas Ihab F., Papotti Paolo. Discovering denial constraints // Proc. VLDB Endow. — 2013. — Vol. 6, no. 13. — P. 1498–1509. — URL: https://doi.org/10.14778/2536258.2536262.

[6] Chu Xu, Ilyas Ihab F., Papotti Paolo. Holistic data cleaning: Putting violations into context // 2013 IEEE 29th International Conference on Data Engineering (ICDE). — 2013. — P. 458–469.

[7] Cleaning inconsistencies in information extraction via prioritized repairs / Ronald Fagin, Benny Kimelfeld, Frederick Reiss, Stijn Vansummeren // Proceedings of the 33rd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems. — PODS '14. — New York, NY, USA : Association for Computing Machinery, 2014. — P. 164–175. — URL: https://doi.org/10.1145/2594538.2594540.

[8] Conditional Functional Dependencies for Data Cleaning / Philip Bohannon, Wenfei Fan, Floris Geerts et al. // 2007 IEEE 23rd International Conference on Data Engineering. — 2007. — P. 746–755.

[9] Desbordante: From Benchmarking Suite to High-Performance Science-Intensive Data Profiler / George A. Chernishev, Mikhail A. Polyntsov, Anton I. Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (CODS-COMAD 2024). — Association for Computing Machinery, 2025. — P. 234–243. — URL: https://doi.org/10.1145/3703323.3703725.

[10] Fan Wenfei, Geerts Floris, Wijsen Jef. Determining the Currency of Data // ACM Trans. Database Syst. — 2012. — Vol. 37, no. 4. — 46 p. — URL: https://doi.org/10.1145/2389241.2389244.

[11] Fast approximate denial constraint discovery / Renjie Xiao, Zijing Tan, Haojin Wang, Shuai Ma // Proc. VLDB Endow. — 2022. — Vol. 16, no. 2. — P. 269–281. — URL: https://doi.org/10.14778/3565816.3565828.

[12] Functional dependency discovery: an experimental evaluation of seven algorithms / Thorsten Papenbrock, Jens Ehrlich, Jannik Marten et al. // Proc. VLDB Endow. — 2015. — Vol. 8, no. 10. — P. 1082–1093. — URL: https://doi.org/10.14778/2794367.2794377.

[13] HoloClean: holistic data repairs with probabilistic inference / Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, Christopher Ré // Proc. VLDB Endow. — 2017. — Vol. 10, no. 11. — P. 1190–1201. — URL: https://doi.org/10.14778/3137628.3137631.

[14] Intel Corporation. Intel Hybrid Architecture. — https://www.intel.com/content/www/us/en/developer/articles/technical/hybrid-architecture.html. — 2023. — Accessed: 2026-03-11.

[15] Pena Eduardo H. M., Porto Fabio, Naumann Felix. Fast Algorithms for Denial Constraint Discovery // Proc. VLDB Endow. — 2022. — Vol. 16, no. 4. — P. 684–696. — URL: https://doi.org/10.14778/3574245.3574254.

[16] Pena Eduardo H. M., de Almeida Eduardo Cunha. BFASTDC: A Bitwise Algorithm for Mining Denial Constraints // Database and Expert Systems Applications / Ed. by Sven Hartmann, Hui Ma, Abdelkader Hameurlain et al. — Cham : Springer International Publishing, 2018. — P. 53–68.

[17] Pena Eduardo H. M., de Almeida Eduardo C., Naumann Felix. Discovery of approximate (and exact) denial constraints // Proc. VLDB Endow. — 2019. — Vol. 13, no. 3. — P. 266–278. — URL: https://doi.org/10.14778/3368289.3368293.

[18] Rahm Erhard, Do Hong Hai. Data Cleaning: Problems and Current Approaches. // IEEE Data(base) Engineering Bulletin. — 2000. — Vol. 23. — P. 3–13. — URL: https://api.semanticscholar.org/CorpusID:260972099.

[19] RangerShaw. FastADC: An Efficient Algorithm for Dependency Constraints. — https://github.com/RangerShaw/FastADC. — 2024. — GitHub repository.

[20] Towards an End-to-End Human-Centric Data Cleaning Framework / El Kindi Rezig, Mourad Ouzzani, Ahmed K. Elmagarmid et al. // Proceedings of the Workshop on Human-In-the-Loop Data Analytics. — HILDA '19. — New York, NY, USA : Association for Computing Machinery, 2019. — 7 p. — URL: https://doi.org/10.1145/3328519.3329133.

[21] Википедия. Гиперпоточность. — https://ru.wikipedia.org/wiki/Гиперпоточность. — 2026. — Дата обращения: 11.03.2026.

[22] Википедия. Одновременная многопоточность. — https://ru.wikipedia.org/wiki/Одновременная_многопоточность. — 2026. — Дата обращения: 11.03.2026.

[23] Морозко Иван Дмитриевич. Реализация алгоритма поиска запрещающих ограничений в платформе Desbordante. — 2025. —

https://github.com/spbu-se/mag_practices_2023-2025/blob/main/ПИ/Морозко%20Иван%20Дмитриевич/semester_2/Morozko_report.pdf.